

AURA

AI-Based Student Mental Wellness & Academic Companion

Complete Project Documentation

From Scratch to Full Implementation & Deployment

Institution:	Aditya College of Engineering and Technology
Tech Stack:	Flask + MongoDB + Google Gemini AI + Jinja2 + Vanilla JS
Total API Routes:	137+
HTML Templates:	16
Database Models:	15
MongoDB Collections:	20+
JavaScript Modules:	9
CSS Files:	10
Version:	1.0 -- March 2026

Generated from complete codebase analysis -- every file read and documented

Table of Contents

AURA -- Complete Project Notes (Scratch to Full Implementation)

TABLE OF CONTENTS

1. PROJECT OVERVIEW & PURPOSE
 2. SYSTEM ARCHITECTURE
 3. TECH STACK & DEPENDENCIES
 4. PROJECT FILE STRUCTURE
 5. CONFIGURATION & ENVIRONMENT
 6. DATABASE DESIGN (MongoDB)
 7. MODELS LAYER
 8. AUTHENTICATION & SECURITY
 9. BACKEND ROUTES -- COMPLETE API REFERENCE
 10. SERVICES LAYER (BUSINESS LOGIC)
 11. UTILITIES LAYER
 12. FRONTEND -- TEMPLATES (HTML)
 13. FRONTEND -- JAVASCRIPT MODULES
 14. FRONTEND -- CSS & THEMING
 15. REAL-TIME FEATURES (WebSocket)
 16. SIGNAL PIPELINE & RISK DETECTION
 17. RBAC & ACCESS CONTROL
 18. RATE LIMITING & BRUTE FORCE PROTECTION
 19. AUDIT LOGGING
 20. DEPLOYMENT
 21. DATA FLOW DIAGRAMS
- QUICK REFERENCE -- COMPLETE ROUTE COUNT

AURA -- Complete Project Notes (Scratch to Full Implementation)

Project: AURA -- AI-Based Student Mental Wellness & Academic Companion

Institution: Aditya College of Engineering and Technology

Tech Stack: Flask + MongoDB + Google Gemini AI + Jinja2 + Vanilla JS

Total Routes: 137+ | Templates: 16 | Models: 15 | JS Modules: 9 | CSS Files: 10

TABLE OF CONTENTS

- * Project Overview & Purpose
- * System Architecture
- * Tech Stack & Dependencies
- * Project File Structure
- * Configuration & Environment
- * Database Design (MongoDB)
- * Models Layer
- * Authentication & Security
- * Backend Routes -- Complete API Reference
 - * 9.1 Auth Routes
 - * 9.2 Student Routes
 - * 9.3 Proctor Routes
 - * 9.4 HOD Routes
 - * 9.5 Parent Routes
 - * 9.6 Chat Routes (AI)
 - * 9.7 Connect Hub Routes
- * Services Layer (Business Logic)
 - 10.1 AI Service
 - 10.2 OTP Service
 - 10.3 Stress Service (Dynamic Engine)
- * Utilities Layer
- * Frontend -- Templates (HTML)
- * Frontend -- JavaScript Modules
- * Frontend -- CSS & Theming
- * Real-Time Features (WebSocket)
- * Signal Pipeline & Risk Detection
- * RBAC & Access Control
- * Rate Limiting & Brute Force Protection
- * Audit Logging
- * Deployment
- * Data Flow Diagrams

1. PROJECT OVERVIEW & PURPOSE

AURA is a closed-loop behavioral telemetry and intervention system for institutional student wellness. It is NOT a generic health app -- it is a four-layer governance system where:

- * Layer 1 (Student): Students interact with wellness tools, chatbots, mood tracking -- positive language ONLY, no risk labels shown
- * Layer 2 (Signal Pipeline): Backend-only hidden evaluation -- detects stress spikes, low mood patterns, distress language, auto-escalation -- student NEVER sees this
- * Layer 3 (Proctor/HOD): Anonymous incident queue -- proctor sees anonymous ID (STU_XXXXX), never real email; HOD sees department-wide analytics and proctor performance
- * Layer 4 (Parent): OTP-authenticated parent portal -- wellness monitoring, academic tracking, complaints, suggestions -- linked to student via phone number

Core Features

- * Mood-Based Adaptive UI -- Theme changes based on student emotions (6 moods: happy, calm, stressed, anxious, angry, sad)
- * AI Mental Health Chatbot -- Google Gemini 2.5 Flash with empathetic responses, crisis interceptor, 5-turn context
- * Multimodal Study Assistant -- Analyzes PDFs/images, generates quizzes, summaries, flashcards
- * 6-Signal Stress Engine -- Mood + Sentiment + Activity + Volatility + Time Bias + Trend -> Logistic compression -> EMA smoothing
- * Proctor Dashboard -- Anonymous student watchlist, incident workflow, case management, audit trail
- * HOD Executive Dashboard -- Department-wide wellness analytics, risk distribution, proctor performance
- * Parent Portal -- OTP-based auth, wellness monitoring, academic tracking, complaint system
- * Connect Hub -- Peer networking, groups, events, resources, DMs, group chat, activity feed
- * Relaxation Toolkit -- Procedural audio synthesis (rain/ocean/forest), meditation timer, binaural beats, breathing exercises
- * Gamification -- 8+ cognitive games with XP system, combo multiplier, difficulty progression
- * Grievance System -- Student issue reporting with proctor resolution workflow
- * Academic Tracker -- CGPA/SGPA/Attendance tracking with at-risk detection

2. SYSTEM ARCHITECTURE

```
+-----+
|                                     |
|               FRONTEND (Browser)  |
| +-----+ +-----+ +-----+ +-----+ +-----+ |
| | Student | | Proctor | | HOD   | | Parent | |Connect | |
| |Dashboard| |Dashboard| |Dashboard| |Dashboard| | Hub   | |
| +-----+ +-----+ +-----+ +-----+ +-----+ |
| |         | |         | |         | |         | |         | |
| +-----+ +-----+ +-----+ +-----+ +-----+ |
| |               REST API (fetch) + WebSocket (Socket.IO) | |
| +-----+ +-----+ +-----+ +-----+ +-----+ |
+-----+
|
+-----+
... (23 more lines)
```

3. TECH STACK & DEPENDENCIES

Backend

Component	Technology	Version
Framework	Flask	3.0.0
Database	MongoDB (PyMongo)	4.7.3
AI Engine	Google Gemini (google-genai)	1.56.0
AI Fallback	OpenAI, Groq, DeepSeek	Latest
Authentication	bcrypt + Session-based	4.1.2
Rate Limiting	Flask-Limiter	4.1.1
Email	Flask-Mail	0.9.1
SMS	Fast2SMS API	via requests
Real-time	Flask-SocketIO	5.3.7
WSGI (Linux)	Gunicorn	23.0.0
WSGI (Windows)	Waitress	--
Environment	python-dotenv	1.0.1

Frontend

Component	Technology
Templates	Jinja2 (Flask)
Charts	ApexCharts 3.35.0, Chart.js 4.4.0
Real-time	Socket.IO 4.7.4
Markdown	Marked.js
Code Highlighting	Highlight.js 11.10.0
Math Rendering	KaTeX 0.16.11
Icons	Font Awesome 6.5.0
Fonts	Google Inter
Audio	Web Audio API (native, procedural)
Graphics	Canvas API, SVG
PWA	Service Worker

4. PROJECT FILE STRUCTURE

```

AURA/
+-- app.py                # Flask app factory -- limiter, SocketIO, security headers, indexes
+-- config.py             # Centralized config (SECRET_KEY, MongoDB, Mail, Session, Rate Limit)
+-- run.py                # Dev entrypoint -- SocketIO on port 5000
+-- wsgi.py               # Production entrypoint -- Gunicorn (Linux) / Waitress (Windows)
+-- requirements.txt      # Python dependencies
|
+-- models/               # MongoDB schema helpers (15 models)
|   +-- __init__.py       # Exports all model classes, init_models()
|   +-- user.py           # User accounts (student/proctor/hod/admin)
|   +-- chat.py           # Chat messages (mental/study)
|   +-- mood.py           # Student mood entries (1-10 intensity)
|   +-- stress.py         # Stress scores (0-100) + calculate_stress()
|   +-- grievance.py      # Grievance tickets (pending/in_progress/resolved)
|   +-- parent.py         # Parent OTP accounts + CRUD methods
|   +-- connect_hub.py    # 9 models: Connection, Group, Event, Resource,
|                           # HubActivity, PeerMessage, GroupMessage, HubFeed,
|                           # HubNotification
|
+-- routes/               # Flask blueprints (6)
|   +-- __init__.py       # init_routes() -- registers all blueprints
|   +-- auth.py           # /login, /logout
|   +-- student.py        # /student/* (~35 endpoints)
|   +-- proctor.py        # /proctor/* (~46 endpoints)
|   +-- parent.py         # /parent/* (~16 endpoints)
|   +-- chat.py           # /api/chat/*, /study/* (~10 endpoints)
|   +-- connect_hub.py    # /student/hub/*, /student/api/connect/* (~30+ endpoints)
|
+-- services/             # Business logic
|   +-- ai_service.py     # Gemini AI + DeepSeek/Groq/OpenAI fallback chain
|   +-- otp_service.py    # Fast2SMS OTP generation, verification, cooldowns
|   +-- stress_service.py # 6-signal dynamic stress engine with EMA + logistic compression
|
+-- utils/                # Shared utilities
|   +-- access_control.py # RBAC scoping, anonymous IDs, visibility checks
|   +-- alerts.py         # Email alerts to proctors/parents on high stress
|   +-- audit_logger.py   # Non-blocking centralized audit logging
|   +-- auth_helpers.py   # @login_required, @demo_restricted, bcrypt hashing
|   +-- database.py       # MongoDB init, get_db(), demo data seeding
|   +-- helpers.py        # Content filtering, safe_error()
|   +-- rate_limit.py     # 5-tier rate limiting + brute-force protection
|   +-- validators.py     # Input validation
|
+-- templates/ (16 files) # Jinja2 HTML templates
+-- static/
|   +-- css/ (10 files)   # Stylesheets
|   +-- js/ (9 files)     # JavaScript modules
|   +-- service-worker.js # PWA offline caching
|   +-- uploads/          # User-uploaded files
|   +-- images/, assets/  # Static assets
|
+-- scripts/tools/         # Dev/test utilities (15 scripts)
+-- docs/                 # Documentation

```

5. CONFIGURATION & ENVIRONMENT

config.py -- Centralized Configuration Class

Setting	Default	Purpose
SECRET_KEY	Random 32-byte hex	Flask session encryption
DEBUG	false	Flask debug mode
SESSION_COOKIE_HTTPONLY	True	Prevent JS access to cookies
SESSION_COOKIE_SAMESITE	Lax	CSRF protection
SESSION_COOKIE_SECURE	env-based	HTTPS-only cookies
PERMANENT_SESSION_LIFETIME	3600s (1 hour)	Session timeout
MAX_CONTENT_LENGTH	16 MB	Upload size limit
MONGODB_URI	mongodb://localhost:27017/	Database connection
MONGODB_DB_NAME	aura_db	Database name
RATELIMIT_STORAGE_URI	memory:// (dev) / redis:// (prod)	Rate limit backend
FAST2SMS_API_KEY	env	SMS gateway for parent OTP
MAIL_SERVER	smtp.gmail.com	Email server
MAIL_PORT	587	SMTP port
PROXY_FIX_ENABLED	false	Reverse proxy header trust

Required Environment Variables (.env)

```
SECRET_KEY=<your-secret>
MONGODB_URI=mongodb+srv://...
GEMINI_API_KEY=<google-ai-key>
MAIL_USERNAME=<email>
MAIL_PASSWORD=<app-password>
FAST2SMS_API_KEY=<sms-key>
# Optional:
OPENAI_API_KEY=<fallback>
GROQ_API_KEY=<fallback>
DEEPSEEK_API_KEY=<fallback>
FLASK_DEBUG=true
```

6. DATABASE DESIGN (MongoDB)

Collections Overview (20+)

Collection	Purpose	Key Fields
users	User accounts	email (unique), hashed_password, name, role, department, roll_number, parent_phone
chats	AI chat messages	user_email, message, response, type (mental/study), sentiment, created_at
moods	Student mood entries	user_email, mood (happy/calm/stressed/anxious/angry/sad), intensity (1-10), created_at
stress	Stress scores	user_email, score (0-100), source, created_at
grievances	Student complaints	user_email, subject, description, status, resolved_by, proctor_note
parents	Parent accounts	student_roll, parent_name, parent_phone, relationship, auth_type (otp)
otp_codes	OTP storage	phone, otp_hash, attempts, created_at, expires_at, verified
student_wellness	Unified wellness data	student_id, data_type (stress/mood/hub_engagement), value, timestamp
student_journals	Daily reflections	student_id, date, entry (max 2000 chars)
risk_incidents	Signal pipeline incidents	incident_id, anonymous_student_id, risk_level, trigger_source, status, case_status
proctor_actions	Incident workflow actions	action_id, proctor_id, incident_id, action_type, old_status, new_status
proctor_notes	Private intervention notes	anonymous_student_id, proctor_id, note, urgent, risk_score
proctor_students	Proctor-student assignments	proctor_id, anonymous_id, roll_number, department, status
proctor_activity_logs	Centralized audit trail	proctor_email, action, target_type, target_id, ip_address, timestamp
support_requests	Help requests	student_id, priority, type (urgent/general), status
counseling_sessions	Booked sessions	student_id, date, time, type, status
academic_records	Semester grades	student_roll, semester, sgpa, cgpa, attendance, backlogs, credits
academic_subjects	Subject marks	student_roll, semester, code, name, internal, external, grade
connections	Peer connections	user_email, connected_to, status (pending/accepted/rejected)
groups	Hub groups	group_id, name, type (study/relaxation/peer_support), members[], member_count
events	Hub events	event_id, title, type (webinar/meditation/workshop), date, participants[]
resources	Shared resources	resource_id, title, link, tags[], likes, liked_by[]
peer_messages	Direct messages	from_email, to_email, message, seen, created_at
group_messages	Group messages	group_id, sender_email, message, created_at
hub_feed	Activity feed	actor_name, action, target, created_at

Collection	Purpose	Key Fields
hub_notifications	Notifications	user_email, type, title, body, read, link
hub_activity	Online heartbeat	user_email, last_active
room_messages	Connection hub chat rooms	room_id, user_email, display_name, message, reported
alerts	Institutional alerts	student_email, score, proctor_email, parent_email, status
parent_complaints	Parent complaints	student_roll, parent_name, category, subject, description, status
parent_suggestions	Parent suggestions	student_roll, parent_name, title, description, status
announcements	Institutional notices	type, title, content, department, created_at
system_status	System state	status, active_students, active_alerts

Index Strategy

- * Unique indexes: users.email, connections(user_email + connected_to), groups.group_id, events.event_id, resources.resource_id
- * Compound indexes: risk_incidents(status + risk_level + timestamp), proctor_actions(proctor_id + timestamp), peer_messages(from_email + to_email + created_at)
- * All indexes created in background on app startup via ensure_production_indexes()

7. MODELS LAYER

15 Model Classes (all in models/)

Each model defines:

- * collection_name -- MongoDB collection string
- * schema() -- Dict of field types (documentation, not enforced by MongoDB)
- * validate() -- Runtime validation for inserts
- * indexes() -- Index specifications (for some models)

Model	Collection	Key Validation
UserModel	users	role ? {student, proctor, hod, admin}, email required
ChatModel	chats	type ? {mental, study}
MoodModel	moods	intensity ? [1, 10]
StressModel	stress	score ? [0, 100], source required
GrievanceModel	grievances	status ? {pending, in_progress, resolved}
ParentModel	parents	Has CRUD static methods: create_parent(), find_by_phone(), find_by_email()
ConnectionModel	connections	status ? {pending, accepted, rejected}, no self-connect

Model	Collection	Key Validation
GroupModel	groups	name 3-60 chars, type ? {study, relaxation, peer_support}, max 20 members
EventModel	events	title 3-100 chars, type ? {webinar, meditation, workshop}
ResourceModel	resources	title 3-120 chars, link must be http(s), max 10 tags
HubActivityModel	hub_activity	Heartbeat tracking for online status
PeerMessageModel	peer_messages	DM schema
GroupMessageModel	group_messages	Group chat schema
HubFeedModel	hub_feed	Feed actions: joined_group, created_group, shared_resource, etc.
HubNotificationModel	hub_notifications	Types: connection_request, group_invite, event_reminder, message

8. AUTHENTICATION & SECURITY

Login Flow (Student/Proctor/HOD)

- * User submits email + password on /login (POST)
- * Brute-force check: check_login_rate(ip, email) -- max 5 attempts per 5 mins, 10-min lockout
- * User lookup in users collection
- * bcrypt verification: verify_password(hash_password, password)
- * On success: Clear rate limits, set session variables, redirect by role
- * On failure: Record failed attempt, flash generic error

Session Variables Set on Login

```

session['user_email']      # User's email
session['user_name']      # Display name
session['user_role']      # student | proctor | hod
session['user_department'] # Department code (AIDL, CSE, etc.)
session['user_roll']      # Roll number (students only)
session['is_demo']        # Boolean -- restricts write operations

```

Parent Login Flow (OTP-Based)

- * Parent enters phone number -> /parent/api/send-otp
- * System validates phone against student records (users.parent_phone)
- * OTP generated (6-digit, cryptographically secure) -> sent via Fast2SMS
- * Parent enters OTP -> /parent/api/verify-otp
- * If existing parent: auto-login -> dashboard
- * If new parent: show registration form -> /parent/api/complete-registration

Security Headers (app.py @after_request)

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
Content-Security-Policy: default-src 'self'; script-src 'self' cdn.jsdelivr.net ...
```

Decorators

Decorator	Purpose
@login_required	Redirects to /login if no session
@demo_restricted	Blocks write operations for demo accounts (returns 403)
@demo_chat_limited	Limits demo accounts to 5 chat messages per session
@role_required(role)	Requires specific role
@proctor_only	In-file decorator requiring proctor/hod role

Demo Accounts

Email	Password	Role
student@aura.edu	password123	student
proctor@aura.edu	password123	proctor
hod@aura.edu	password123	hod

9. BACKEND ROUTES -- COMPLETE API REFERENCE

9.1 Auth Routes (Blueprint: auth, prefix: /)

Method	URL	Function	Purpose
GET/POST	/login	login()	Login page + authentication
GET	/logout	logout()	Clear session, redirect to login

9.2 Student Routes (Blueprint: student, prefix: /student)

Page Routes

Method	URL	Template	Purpose
GET	/dashboard	student_dashboard.html	Student wellness dashboard
GET	/chat/mental	mental_chatbot.html	AI mental health chatbot
GET	/chat/study	study_chatbot.html	AI study assistant
GET	/relax	relax.html	Relaxation toolkit (6 tools)
GET	/activities	activities.html	Interactive wellness activities (8 tools)
GET	/games	games.html	Cognitive training games with XP
GET	/_unregister_sw	unregister_sw.html	Service worker management

Wellness API

Method	URL	Purpose
GET	/api/wellness/current	Live-computed stress with 6 signals, trend, confidence
GET	/api/wellness/activities	Activity counts (today, week, weekly average, change %)
POST	/api/wellness/checkin	Submit mood (1-5) + stress (0-100) + notes -> triggers hidden signal pipeline
GET	/api/wellness/goals	Goal completion (checkin, breathing, study, relax)

Mood & Stress API

Method	URL	Purpose
GET	/api/mood/today	Check if mood set today
POST	/api/mood	Update mood -> recalculate stress
GET	/api/stress/today	Today's stress computed live
GET	/api/student/stress-level	Stress + weekly stats + trend analysis
GET	/api/stress_history	Bucketed by day (7-90 days)

Support Center

Method	URL	Purpose
POST	/api/support/urgent	Crisis help -> creates HIGH priority proctor incident
POST	/api/support/schedule	Book counseling session
GET	/api/support/sessions	My booked sessions
POST	/api/support/request	Request support -> MEDIUM priority incident

Profile & Account

Method	URL	Purpose
GET	/api/student/profile	Profile info from session
POST	/api/student/change-password	Change password (validates current)

Dashboard Data

Method	URL	Purpose
GET	/api/student/dashboard-data	Mood, streak, activities, AI insight
GET	/api/activities/count	Total activity count

Journal

Method	URL	Purpose
POST	/api/journal	Save reflection (max 2000 chars, one per day)
GET	/api/journal/today	Get today's journal

Quick Actions

Method	URL	Purpose
POST	/api/quick_actions	Handle breathing/stretch/energy_boost/wind_down -> reduces stress score

Connection Hub Rooms

Method	URL	Purpose
GET	/api/connection/rooms	List chat rooms (campus_life, exam_stress, placements, etc.)
GET	/api/connection/rooms/<room_id>/messages	Last 50 messages (chronological)
POST	/api/connection/rooms/<room_id>/send	Send message (profanity + rate limit check)
POST	/api/connection/messages/<id>/report	Report message for proctor review

Academic Performance

Method	URL	Purpose
GET	/api/student/academics	Semester records + subjects (seeds demo data if needed)

9.3 Proctor Routes (Blueprint: proctor, prefix: /proctor)

Page Routes

Method	URL	Template	Purpose
GET	/dashboard	proctor_dashboard.html	Proctor command center
GET	/student/<anonymous_id>	student_detail.html	Anonymous student profile

Student Management

Method	URL	Purpose
POST	/api/student/add	Add student to ward (creates user + parent accounts)
GET	/api/student/<id>/details	Full intervention profile (incidents, actions, notes, sessions, stats)
GET	/api/my-students	All assigned students with wellness status
POST	/api/my-students/<id>/remove	Soft-delete student from ward

Case Workflow

Method	URL	Purpose
PATCH	/api/case/<id>/status	Update case status: new -> reviewing -> assigned -> contacted -> monitoring -> resolved
POST	/api/case/<id>/assign	Assign counselor to case

Proctor Notes

Method	URL	Purpose
GET	/api/notes/<anonymous_id>	Get private intervention notes
POST	/api/notes/<anonymous_id>	Add note (optional: urgent flag, risk_score, follow_up_date, flag_monitoring)

Dashboard Metrics

Method	URL	Purpose
GET	/api/dashboard/summary	KPIs: pending by risk, resolved, today count, auto-escalations, grievances
GET	/api/metrics/resolution	Resolution rate %, avg resolution time, pending HIGH risk
GET	/api/system/status	System operational state
GET	/api/health	Health check (pending incidents, last activity)

Risk Queue

Method	URL	Purpose
GET	/api/risk/queue	Filtered incident queue (status, risk_level, time_range, sort)
GET	/api/risk/queue/time/<range>	Queue by time range (hour/24h/7d)
GET	/api/risk/search	Search incidents by ID/room/message (regex, min 3 chars)
GET	/api/incidents/<id>	Incident details + action history

Incident Actions

Method	URL	Purpose
POST	/api/action/<type>	Single action: dismiss/remove/escalate/contact/monitor/close/review
POST	/api/action/bulk	Bulk action on multiple incidents

Audit & Logging

Method	URL	Purpose
GET	/api/audit/logs	Proctor action audit trail (up to 90 days)
GET	/api/audit/export/csv	Export audit as CSV
GET	/api/activity-logs	Centralized activity logs with action summary
GET	/api/activity-logs/export/csv	Export activity logs CSV (up to 365 days)

Academic Management

Method	URL	Purpose
GET	/api/academics/overview	All students' academic performance
GET	/api/academics/student/<id>	Single student academic profile
POST	/api/academics/student/<id>/record	Add/update semester record (auto-detects CGPA drops)
POST	/api/academics/subjects	Add subject marks
GET	/api/academics/at-risk	At-risk students (CGPA < 5, attendance < 65%, backlogs >= 2)
GET	/api/academics/department-stats	CGPA distribution, pass rate, average attendance

Support Center Management

Method	URL	Purpose
GET	/api/support/tickets	Get support tickets (filterable by type/status)
PATCH	/api/support/tickets/<id>/status	Update ticket status
GET	/api/support/sessions	Counseling session list
PATCH	/api/support/sessions/<id>/status	Update session status
GET	/api/support/stats	Support center workload stats

Grievances

Method	URL	Purpose
GET	/api/grievances	RBAC-scoped grievance list
PATCH	/api/grievances/<id>/status	Update grievance status + resolution note

9.4 HOD Routes (within proctor blueprint, prefix: /proctor)

Method	URL	Purpose
GET	/hod	HOD executive dashboard page
GET	/api/hod/dashboard-stats	Active students, incidents, wellness index, resolution rate
GET	/api/hod/wellness-trends	30-day department-wide stress averages by day
GET	/api/hod/risk-distribution	Incident count by HIGH/MEDIUM/LOW
GET	/api/hod/proctor-performance	Proctor action counts (dismissals, escalations, removals)
GET	/api/hod/recent-escalations	Latest 20 escalated incidents

9.5 Parent Routes (Blueprint: parent, prefix: /parent)

Authentication (OTP)

Method	URL	Purpose
GET	/login	Parent login page
GET	/register	Redirects to /parent/login
POST	/api/send-otp	Validate phone + send OTP via SMS
POST	/api/verify-otp	Verify OTP -> auto-login or register
POST	/api/complete-registration	Complete parent profile after OTP
GET	/logout	Parent logout

Dashboard & APIs

Method	URL	Purpose
GET	/dashboard	Parent dashboard page
GET	/api/student/performance	Stress + mood trends (30 days)
GET	/api/student/academics	CGPA, SGPA, attendance, credits by semester
GET	/api/student/wellness-summary	Comprehensive wellness summary with status
GET	/api/student/activity-log	Recent activities (last 50)
POST	/api/complaint/submit	Submit complaint (category, priority)
GET	/api/complaints/list	Parent's complaints
POST	/api/suggestion/submit	Submit suggestion
GET	/api/student/wellness-summary	Comprehensive wellness summary with status
GET	/api/student/activity-log	Recent student activities (last 50)
GET	/api/announcements	Department achievements/placements
GET	/api/notifications	Parent notifications

9.6 Chat Routes (Blueprint: chat, prefix: /)

Method	URL	Purpose
POST	/api/chat/mental	Mental health chat -> Gemini AI with sentiment analysis
POST	/api/chat	Unified chat endpoint (proxies to mental)
GET	/api/chat/history	Paginated chat history (max 200)
POST	/api/chat/clear	Delete all mental chats (demo_restricted)
POST	/upload_study_file	Upload study material (PDF, PNG, JPG, DOC)
POST	/study/upload	Preferred upload route
POST	/study/summarize	AI-summarize uploaded file
POST	/study/quiz	Generate 5-question quiz from file
POST	/api/study/analyze	Analyze query ± uploaded file
POST	/api/chat/feedback	Telemetry (thumbs up/down/copy)

9.7 Connect Hub Routes (Blueprint: connect, prefix: /student)

Page

Method	URL	Purpose
GET	/hub or /hub/<path>	Connect Hub SPA page (auto-seeds on first visit)

Peer Network

Method	URL	Purpose
GET	/api/connect/peers	List accepted friends + online status + unread DMs
GET	/api/connect/requests	Pending connection requests (incoming + outgoing)
POST	/api/connect/request	Send connection request
POST	/api/connect/respond	Accept/reject request
GET	/api/connect/suggestions	AI-scored peer recommendations (stress proximity, dept, groups, engagement)
POST	/api/connect/remove	Remove connection

Groups

Method	URL	Purpose
GET	/api/groups	List groups (paginated, filterable by type)
POST	/api/groups/create	Create group (auto-selects type based on stress)
POST	/api/groups/join	Join group (max 20 members)
POST	/api/groups/leave	Leave group
GET	/api/groups/<id>/members	Group members + online status

Events

Method	URL	Purpose
GET	/api/events	List upcoming events
POST	/api/events/create	Create event (webinar/meditation/workshop)
POST	/api/events/rsvp	Register for event
POST	/api/events/cancel	Cancel registration
GET	/api/events/<id>/ics	Export as .ics calendar file

Resources

Method	URL	Purpose
GET	/api/resources	List resources (popular/recent, tags filter)
POST	/api/resources	Share new resource (sanitized URL, max 10 tags)
POST	/api/resources/like	Like/unlike toggle

Direct Messaging

Method	URL	Purpose
GET	/api/chat/dm/<email>	Get DM thread (marks as read)
POST	/api/chat/dm/<email>/send	Send DM (profanity + rate limit)
GET	/api/chat/dm/unread	Unread counts by peer

Group Chat

Method	URL	Purpose
GET	/api/chat/group/<id>	Get group messages
POST	/api/chat/group/<id>/send	Send group message

Feed, Notifications, Stats

Method	URL	Purpose
GET	/api/connect-hub/feed	Community activity feed
GET	/api/connect-hub/notifications	User notifications + unread count
POST	/api/connect-hub/notifications/read	Mark notification(s) as read
GET	/api/connect-hub/stats	Dashboard snapshot (active now, groups, events, etc.)
GET	/api/connect-hub/recommendations	AI contextual recommendations based on stress/engagement

10. SERVICES LAYER (BUSINESS LOGIC)

10.1 AI Service (services/ai_service.py)

Provider Chain: Gemini -> DeepSeek -> Groq -> OpenAI -> Local Fallback

Key Functions

Function	Purpose
generate_mental_response()	Mental wellness chat with empathetic prompting, crisis interceptor
generate_study_response()	Academic query answering with study-tuned prompts
analyze_study_material()	Multimodal analysis (PDF/image) via Gemini vision
extract_sentiment()	Keyword-based sentiment: anxious/negative/positive/neutral
_classify_request()	Determines response style: ultra_brief/concise/structured
_local_fallback()	Hardcoded responses when all APIs down

Crisis Interceptor: Detects distress keywords in mental chat and prepends crisis resources (988 Lifeline).

Response Styles:

- * ultra_brief -- Greetings, simple queries (<20 chars)
- * concise -- Default conversational
- * structured -- Detailed with sections when user asks for detail

10.2 OTP Service (services/otp_service.py)

Function	Purpose
generate_otp()	6-digit cryptographically random OTP
normalize_phone()	Strips country code (91), leading zero -> 10 digits
send_otp()	Generate + store + send via Fast2SMS (30s resend cooldown)
verify_otp()	Validate OTP (5-min expiry, max 3 attempts)
is_phone_verified()	Check if phone verified within 10 minutes
_send_sms()	Fast2SMS HTTP API call

Demo Mode: If SMS not sent (no API key / failure), OTP returned in response for testing.

10.3 Stress Service (services/stress_service.py) -- THE CORE ENGINE

6-Signal Dynamic Stress Calculation Pipeline:

```

+-----+ +-----+ +-----+
| Signal 1 | | Signal 2 | | Signal 3 |
| MOOD (35%) | | SENTIMENT | | ACTIVITY |
| Latest mood | | (25%) Chat | | (15%) Yerkes- |
| in 24h + | | sentiment | | Dodson curve |
| temporal | | weighted | | (optimal |
| decay | | exponential | | 5-8 actions) |
+-----+ +-----+ +-----+
| | | |
+-----+ +-----+ +-----+
| Signal 4 | | Signal 5 | | Signal 6 |
| VOLATILITY | | TIME BIAS | | TREND |
| (10%) Mood | | (5%) Late- | | (10%) 7-day |
| std dev | | night | | weighted |
| in 48h | | penalty | | direction |
+-----+ +-----+ +-----+
| | | |
+-----+ +-----+
| |
+-----+ +-----+
| Adaptive Weights | <- Redistributes weight from
| (handles missing | missing signals to present ones
| signals) |
+-----+ +-----+
| |
+-----+ +-----+
| Weighted Sum |
| + EMA Smoothing | <- Exponential Moving Average
+-----+ +-----+ prevents oscillation
| |
+-----+ +-----+
| Logistic | <-  $S = 100 / (1 + e^{-(k \cdot (S - ?))})$ 
| Compression | Prevents extreme scores
+-----+ +-----+
| |
+-----+ +-----+
| Z-Score Spike | <- Detects > 2 std dev anomalies
| Detection |
+-----+ +-----+
| |
+-----+ +-----+
| Final Score | -> 0-100 with label:
| + Confidence | Relaxed <=25 | Manageable <=45
| + Insight | Elevated <=65 | High <=80
+-----+ +-----+ Critical >80

```

Stress Labels:

Score Range	Label
0-25	Relaxed
26-45	Manageable
46-65	Elevated
66-80	High
81-100	Critical

Output Object:

```
{
  'score': 62,          # Final compressed score
  'label': 'Elevated',  # Human-readable label
  'trend': 'increasing', # 7-day direction
  'signals': {          # Individual signal scores
    'mood': 70, 'sentiment': 55, 'activity': 40,
    'volatility': 65, 'time_bias': 30, 'trend': 60
  },
  'spike_detected': False, # Z-score anomaly
  'insight': '...',        # Human-readable explanation
  'confidence': 0.78,      # 0.0-1.0 data quality
  'dominant_factor': 'mood', # Highest weighted signal
  'explanation': '...',     # Why this score
  'updated_at': '2026-...'
}
```

11. UTILITIES LAYER

utils/access_control.py -- RBAC

- * create_anonymous_id(email) -> STU_{MD5(email)%100000:05d} (canonical, single source of truth)
- * get_visible_student_ids(user) -> List of anonymous IDs based on role
- * can_access_student(anonymous_id, user) -> Boolean permission check
- * get_incident_filter(user) -> MongoDB query filter for role-scoped incidents

utils/auth_helpers.py -- Authentication

- * hash_password() / verify_password() -- bcrypt
- * @login_required -- Requires session
- * @demo_restricted -- Blocks writes for demo accounts
- * @demo_chat_limited -- Max 5 chats for demos
- * Demo accounts: student@aura.edu, proctor@aura.edu, hod@aura.edu (password123)

utils/database.py -- MongoDB

- * init_db() -- Initialize connection + indexes + seed demo data
- * get_db() -- Get active connection (auto-init if needed)
- * seed_demo_data() -- Creates 3 demo users + sample data

utils/rate_limit.py -- Rate Limiting

5+2 Tiers:

Tier	Limit	Use
STRICT	5/min	Login, password, OTP
MODERATE	30/min	Write operations
STANDARD	60/min	Read APIs
RELAXED	120/min	Dashboard pages
EXPORT	10/min	CSV downloads
BULK	10/min	Bulk operations
SEARCH	30/min	Search queries

Login Protection: 5 attempts per 5 mins -> 10-minute logout

utils/alerts.py -- Notifications

- * Sends email alerts to proctor + parent when stress exceeds threshold
- * Logs to alerts collection

utils/audit_logger.py -- Audit Trail

- * log_activity(action, target_type, target_id, metadata) -- Non-blocking insert to proctor_activity_logs
- * 15+ action types: LOGIN, LOGOUT, ADD_STUDENT, REVIEW_INCIDENT, ESCALATE_INCIDENT, etc.

utils/helpers.py -- Content Safety

- * contains_blocked_content(text) -- Word-boundary regex for 10 blocked terms
- * safe_error(exc, context) -- Sanitized error messages (debug vs production)

utils/validators.py -- Input Validation

- * validate_incident_action(data) -- Validates action payloads

12. FRONTEND -- TEMPLATES (HTML)

Template Inheritance

```
base.html (master layout)
+-- student_dashboard.html (extends)
+-- games.html (extends)
+-- unregister_sw.html (extends)

13 standalone templates (full HTML, no inheritance):
login.html, index.html, mental_chatbot.html, study_chatbot.html,
relax.html, activities.html, connect_hub.html, proctor_dashboard.html,
hod_dashboard.html, student_detail.html, parent_login.html,
parent_dashboard.html
```

Template Summary

Template	Type	Theme	Key Features
base.html	Master layout	Dynamic	Navbar, mood indicator, Zen Mode, service worker
login.html	Public	Dark/Aurora	2 canvas animations (Perlin noise + neural network), demo account cards
student_dashboard.html	Student	Dynamic	3-tab system, stress signals, ApexCharts, quick actions
mental_chatbot.html	Student	Dark	Word-reveal animation, crisis banner (988), mood buttons, KaTeX
study_chatbot.html	Student	Dark	File upload, quiz/summary/flashcard generation, code highlighting
relax.html	Student	Dark	6 tools: ambient sounds (Web Audio synthesis), meditation, binaural beats
activities.html	Student	Dark	8 tools: scream meter (microphone), canvas drawing, breathing, pomodoro
games.html	Student	Dark	8+ games with XP/combo system, confetti animations
connect_hub.html	Student	Dynamic	Full SPA: peers, groups, events, resources, DMs, feed, notifications
proctor_dashboard.html	Proctor	Light	KPI cards, risk queue, student table, Add Student modal
hod_dashboard.html	HOD	Light	ApexCharts donut, risk distribution, proctor performance
student_detail.html	Proctor	Light	Case management, Chart.js trigger breakdown, proctor notes
parent_login.html	Public	Purple	3-step OTP wizard (phone -> OTP -> profile)
parent_dashboard.html	Parent	Light	Wellness + academics + complaints + announcements

13. FRONTEND -- JAVASCRIPT MODULES

9 JavaScript Files

File	Purpose	API Calls	Dependencies
chat-engine.js	Mental health chatbot engine	POST /api/chat/mental	marked.js
study_chatbot.js	Study assistant engine	POST /api/study/analyze, /study/upload	marked, hljs
student_dashboard.js	Student dashboard logic	~10 endpoints	ApexCharts, luxon

File	Purpose	API Calls	Dependencies
connect_hub.js	Connect Hub SPA (largest)	20+ endpoints	socket.io
proctor_dashboard.js	Proctor dashboard	6 endpoints	--
parent_dashboard.js	Parent dashboard	8 endpoints	ApexCharts
mood_handler.js	Mood theme system	POST /student/api/mood	--
theme-engine.js	Light/dark mode	--	--
main.js	Header auto-hide	--	--

Note: service-worker.js is located at static/service-worker.js (root of static, not in js/) and handles PWA offline caching.

Key Frontend Patterns

- * SPA Routing: Connect Hub uses history.pushState() for client-side navigation
- * Polling: 30-second intervals for real-time dashboard updates
- * WebSocket: Socket.IO only in Connect Hub for DMs and typing indicators
- * Markdown: All AI responses rendered through marked.js
- * Theme System: CSS custom properties toggled by mood_handler.js and theme-engine.js
- * localStorage: Used for chat persistence, game scores, theme preferences
- * Skeleton Loading: Placeholder UI while API calls in progress
- * Animation: Count-up animations, word-reveal, confetti bursts, canvas particles

14. FRONTEND -- CSS & THEMING

10 CSS Files

File	Scope	Theme
global.css	All pages	Typography, CSS variables, resets
style.css	Authenticated pages	Cards, activities, components
login.css	Login page	Aurora/neon dark theme
student_dashboard.css	Student dashboard	Responsive grid, signal bars
mental-chatbot.css	Mental chatbot	Dark mode chat bubbles
study-assistant.css	Study chatbot	Dark indigo/teal theme
connect_hub.css	Connect Hub SPA	Full SPA styles
proctor_dashboard.css	Proctor dashboard	Premium clean design system
chat.css	Standalone chat UI	Sidebar + messages
sidebar.css	Standalone chat sidebar	History panel

Mood-Based Theming (6 themes)


```
/* Theme variables change based on active mood */
--aura-primary: #4FC3F7; /* happy -> blue */
--aura-primary: #81C784; /* calm -> green */
--aura-primary: #FF8A65; /* stressed -> orange */
--aura-primary: #CE93D8; /* anxious -> purple */
--aura-primary: #EF5350; /* angry -> red */
--aura-primary: #78909C; /* sad -> gray */
```

15. REAL-TIME FEATURES (WebSocket)

Flask-SocketIO Integration

- * Transport: Socket.IO with threading async mode
- * CORS: Configurable via CORS_ORIGINS env var
- * Used In: Connect Hub (DMs, group chat, typing indicators, online status)
- * Also Used: Auto-escalation alerts to proctor dashboard

Events

Event	Direction	Purpose
peer:online	Server -> Client	Peer came online
peer:typing	Client ? Server	Typing indicator
message:new	Server -> Client	New DM/group message
proctor_alert	Server -> Client	Critical stress auto-escalation

16. SIGNAL PIPELINE & RISK DETECTION

Four-Layer Privacy Architecture

```
STUDENT LAYER (positive language only)
  ? wellness check-in (mood + stress + notes)
SIGNAL PIPELINE (backend-only, student never sees)
  ? evaluates 4 signal types -> creates anonymous incidents
PROCTOR/HOD LAYER (anonymous IDs only, never real email)
  ? HOD sees department-wide analytics + proctor performance
PARENT LAYER (OTP-authenticated, sees child's wellness + academics)
```

4 Signal Types Evaluated

Signal	Trigger	Priority	Logic
Stress Spike	>30 pt jump in 48h	HIGH	Compares oldest stress from 48h ago vs current
Low Mood Pattern	3+ entries <=2 in 48h	MEDIUM	Counts consecutive low mood entries
Distress Language	Stress >=85 + keywords	HIGH	Keyword detection (help, crisis, hopeless, harm)
Critical Auto-Escalation	Stress >85	AUTO/HIGH	6-hour cooldown to prevent alert spam

Incident Status Flow

UNREVIEWED -> REVIEWED -> DISMISSED / ESCALATED / RESOLVED
?

Case Status: new -> reviewing -> assigned -> contacted -> monitoring -> resolved

17. RBAC & ACCESS CONTROL

Role Hierarchy

Role	Can See	Can Do
student	Own data only	Wellness, chat, games, hub
proctor	Assigned students (via proctor_students)	Manage incidents, notes, academics
hod	All students in department	View analytics, proctor performance
parent	Own child's data (via student_roll + phone)	View wellness/academics, submit complaints/suggestions
admin	Everything	Full access

Note: Parent is not stored in the users collection. Parents have their own parents collection and use OTP-based authentication (phone), not password. Session key: parent_logged_in.

Anonymous ID System

- * Formula: STU_ + MD5(email) % 100000 (zero-padded to 5 digits)
- * Example: student@aura.edu -> STU_12345
- * Used in: risk_incidents, proctor_students, proctor_notes, proctor UI
- * Purpose: Proctor never sees real student identity

18. RATE LIMITING & BRUTE FORCE PROTECTION

Login Protection

- * Max 5 attempts per IP+email combination in 5-minute window
- * 10-minute lockout after exceeding
- * Clear on success -- resets after valid login

API Rate Tiers

Tier	Limit	Routes
STRICT	5/min	Login, password change, OTP
MODERATE	30/min	Add student, update case, add note
STANDARD	60/min	Read APIs, audit logs
EXPORT	10/min	CSV downloads
BULK	10/min	Bulk incident actions

Implementation

- * Primary: Flask-Limiter (Redis in production, memory in dev)
- * Fallback: In-memory sliding window (when Flask-Limiter unavailable)
- * Response: HTTP 429 with Retry-After + X-RateLimit-* headers

19. AUDIT LOGGING

Two Audit Systems

- * proctor_actions -- Incident-specific action trail
 - * Fields: action_id, proctor_id, incident_id, action_type, old_status, new_status, reason
 - * Used for: Resolution time calculation, action history on incidents
- * proctor_activity_logs -- Centralized activity audit
 - * Fields: proctor_email, proctor_name, action, target_type, target_id, ip_address, user_agent, metadata, timestamp
 - * Used for: HOD oversight, compliance reporting, CSV export
 - * Actions: LOGIN, LOGOUT, ADD_STUDENT, REMOVE_STUDENT, REVIEW_INCIDENT, DISMISS, ESCALATE, CLOSE, CONTACT, MONITOR, CASE_STATUS_CHANGE, ASSIGN_COUNSELOR, BULK_ACTION, ADD_NOTE, UPDATE_TICKET

CSV Export

- * Proctor audit: /proctor/api/audit/export/csv (max 1000 rows, up to 7 days)
- * Activity logs: /proctor/api/activity-logs/export/csv (max 5000 rows, up to 365 days)

20. DEPLOYMENT

Development (Local)

```
# Install dependencies
pip install -r requirements.txt

# Create .env file with required keys
# Start MongoDB locally or use Atlas connection string

# Run
python run.py
# -> http://localhost:5000
```

Production (Linux -- Gunicorn)

```
gunicorn --worker-class eventlet -w 1 --bind 0.0.0.0:5000 wsgi:app
```

Production (Windows -- Waitress)

```
python wsgi.py
# -> Uses waitress with 8 threads
```

Render Deployment

- * Procfile: Defines gunicorn command
- * render.yaml: Render service configuration
- * start.sh: Linux startup script

Environment Requirements

- * Python 3.12+
- * MongoDB 4.4+ (local or Atlas)
- * .env file with all API keys

21. DATA FLOW DIAGRAMS

Student Wellness Check-in Flow

```
Student clicks "Check In" on dashboard
-> POST /student/api/wellness/checkin { mood: 4, stress: 65, notes: "..."}
-> Inserts to student_wellness collection
-> Bridge writes to moods + stress collections (backward compat)
-> evaluate_risk_signals() [HIDDEN from student]
  -> check_stress_spike() -- >30 pt jump in 48h?
  -> check_low_mood_pattern() -- 3+ low moods in 48h?
  -> check_distress_language() -- stress >=85 + keywords?
  -> auto_escalate_critical_stress() -- stress >85 with 6h cooldown?
  -> IF triggered: create_proctor_incident(anonymous_id, trigger, priority)
    -> Inserts to risk_incidents with anonymous_student_id
    -> Emits socket alert to proctor (if escalated)
-> Returns { success: true, mood: "happy", stress: 65 }
[Student sees ONLY positive confirmation, NEVER risk labels]
```

AI Chat Flow

```
Student types message in mental chatbot
-> POST /api/chat/mental { message: "I feel stressed about exams" }
-> Load last 20 chats from DB for context
-> _classify_request() -> determines response style
-> generate_mental_response()
  -> Build prompt with empathetic system instruction
  -> Try Gemini -> DeepSeek -> Groq -> OpenAI -> local fallback
  -> Crisis interceptor checks for distress keywords
-> extract_sentiment(message + response) -> "anxious"
-> Save to chats collection
-> calculate_dynamic_stress(user_email) -> recalculate stress
-> Return { ai_response, sentiment, stress_update }
```

Proctor Incident Workflow

```
Proctor opens dashboard
-> GET /proctor/api/dashboard/summary -> KPIs
-> GET /proctor/api/risk/queue -> Anonymous incidents

Proctor clicks incident STU_12345
-> GET /proctor/api/incidents/{incident_id} -> Details + action history

Proctor reviews and takes action
-> POST /proctor/api/action/review -> status: REVIEWED, case: reviewing
-> POST /proctor/api/case/{id}/assign -> counselor assigned
-> POST /proctor/api/notes/{anonymous_id} -> add intervention note
-> POST /proctor/api/action/close -> status: RESOLVED, case: resolved

All actions logged to proctor_actions + proctor_activity_logs
```

Parent OTP Flow

```
Parent enters phone
-> POST /parent/api/send-otp { phone: "9876543210" }
  -> Normalize phone (strip country code)
  -> Find student with parent_phone in users collection
  -> Generate 6-digit OTP -> store in otp_codes
  -> Send via Fast2SMS (or return demo_otp if SMS disabled)
  -> Return { masked_phone: "98*****10", student_name }

Parent enters 6-digit OTP
-> POST /parent/api/verify-otp { phone, otp }
  -> Check max attempts (3), expiry (5 min)
  -> If existing parent -> auto-login, redirect to dashboard
  -> If new parent -> prompt registration form

New parent completes profile
-> POST /parent/api/complete-registration { parent_name, relationship }
  -> Verify phone was recently verified (10 min window)
  -> Create parent account (no password -- OTP auth only)
  -> Auto-login -> redirect to dashboard
```

QUICK REFERENCE -- COMPLETE ROUTE COUNT

Blueprint	Prefix	Pages	API Routes	Total
auth	/	2	0	2
student	/student	7	~28	~35
proctor	/proctor	3	~43	~46
parent	/parent	2	~10	~12
chat	/	0	~10	~10
connect	/student	1	~30	~31
TOTAL		15	~121	~137

Notes generated from complete codebase analysis -- every file read from models, routes, services, utils, templates, and JS modules.